Midterm Exam

- Due Mar 11 at 10pm
- Points 100
- Questions 17
- Available Mar 10 at 12am - Mar 11 at 11:59pm
- Time Limit 75 Minutes

Instructions

Partial credit may be given for partially correct solutions.

Use correct C syntax for writing code.

You are not required to write comments for your code; however, brief comments may help make your intention clear in case your code is incorrect.

This quiz was locked Mar 11 at 11:59pm.

Attempt History

	Attempt	Time	Score
LATEST	Attempt 1	74 minutes	70 out of 100

ⓘ Correct answers are hidden.

Score for this quiz: 70 out of 100
Submitted Mar 10 at 4:18pm
This attempt took 74 minutes.

Each of the 6 problems that follow depend on the code given here. Each problem is independant, so changes made to data in one problem do not persist into other problems.

For each problem, type your answer into the textbox. Canvas will auto-grade your responses, so please be careful in typing your answer as only exact answers will be marked correct. The teaching staff will examine all answers marked incorrect and return partial or full credit, as deserved.

For each problem, enter either:

- The word **error** if the code contains any error (compile-time or run-time);
- The words **no output** if the code produces no output; or
- The precise output, including correct spacing, capitalization, and punction.

```
#include <stdio.h>
```

```
enum {
  westley, buttercup, inigo, vizzini, humperdinck, max
};

char *name[] = {
/* 000000000011111111 */
/* 012345678901234567 */
  "Westley",
  "Buttercup",
  "Inigo Montoya",
  "Vizzini",
  "Prince Humperdinck",
  "Miracle Max"
};

char *quote[] = {
/* 00000000001111111111222222222233333333 */
/* 01234567890123456789012345678901234567 */
  "You guessed wrong.",
  "What about the R.O.U.S.es?",
  "You killed my father.  Prepare to die.",
  "Plato, Aristotle, Socrates?  Morons.",
  "Skip to the end.",
  "Have fun storming the castle!"
};
```

Question 1

6 / 6 pts

```
  printf("%s", *(quote + buttercup + vizzini));
```

Skip to the end.

Incorrect

Question 2

0 / 6 pts

```
  char *s = name[inigo];
  printf("%s", s[1] + 1);
```

o

Incorrect

Question 3

0 / 6 pts

```
  printf("%s", quote[inigo] + 23);
```

error

Incorrect

Question 4

0 / 6 pts

```
char **s = name + max;
printf("%s", s[-4] + 6);
```

error

Incorrect

Question 5

0 / 6 pts

```
while (*name[humperdinck]++)
  ;
printf("%s", name[humperdinck] - 1);
```

error

Question 6

20 / 20 pts

Implement the function **last_common**, described below. You may not use any other functions (e.g., from the standard library or otherwise assumed) except, if necessary, **malloc** and **free**. You may not use any non-local variables. You may not write or use any "helper" functions. You may not leak memory. You may assume that all arguments are valid and non-NULL.

last_common returns the last index in strings **s** and **t** wherein both strings contain the same character. If **s** and **t** contain no characters in common, **last_common** returns -1.

Examples:

- s = "foo", t = "bar", return -1; and
- s = "foo", t = "foobar", return 2.

```
int last_common(const char *s, const char *t);
```

Your Answer:

int last_common(const char *s, const char *t){

```
int i = 0, last_count = -1;

while(s[i] && t[i]){

if (s[i] == t[i]) {

last_count = i;

}

i++;

}

return last_count;

}
```

Question 7
20 / 20 pts

Implement the function **dup_smallest**, described below. You may not use any other functions (e.g., from the standard library or otherwise assumed) except, if necessary, **malloc**, **free**, and **memcpy**. You may not use any non-local variables. You may not write or use any "helper" functions. You may not leak memory; it is not a leak to return an address allocated by **malloc** from a function defined to return an address allocated by **malloc**. You may assume that all arguments are valid and non-NULL.

dup_smallest returns a pointer to a newly-**malloc**ed copy of the smaller of **v** and **w** as determined by **compare**. If **v** and **w** are equal, it does not matter which one is copied.

The size in bytes of **v** and **w** are given by **size**.

compare is a standard comparitor. That is, it returns negative if the first parameter is smaller than the second, zero if they are equal, and positive if the first parameter is larger.

You may assume that **v** and **w** do not overlap (a requirement of **memcpy**) and use **memcpy** to do the copy. The **memcpy** function copies **n** bytes from memory area **src** to memory area **dest** and returns **dest**.

```
void *memcpy(void *dest, const void *src, size_t n);
```

```
void *dup_smallest(const void *v, const void *w, size_t size, int (*compare)(const void *, const void *));
```

Your Answer:

```
void *dup_smallest(const void *v, const void *w, size_t size, int (*compare)(const void *, const void *)){
// "The size in bytes of v and w are given by size" - I am assuming that they are the same size
// and thus we can just use this size variable for memcpy.

void *r= malloc(size);
if(compare(v, w) > 0){
// +ve if the first paramater is larger. W is smaller in this case.
memcpy(r, w, size); // Auto updates the value in the pointer r.
} else {
// V is smaller in this case.
memcpy(r, v, size);
}
return r;
}
```

For each of the remaining problems, select **false** if the given code snippet either does not compile or will crash or invoke undefined behavior when executed; otherwise select **true**.

Assume that **int**s are 4 bytes.

Question 8

3 / 3 pts

```
int *s = "hello wirld";
s[7] = 'o';
```

- True
- False

Question 9

3 / 3 pts

```
char *s;
s = malloc(strlen("foo!"));
strcpy(s, "foo!");
```

- True
- False

Question 10

3 / 3 pts

```
void *v = malloc(10);
free(v);
```

- True

False

Question 11

3 / 3 pts

```
void *v = malloc(10);
v++;
free(v - 1);
```

True

False

Question 12

3 / 3 pts

```
int i;
strcpy((char *) &i, "foo");
```

True

False

Question 13

3 / 3 pts

```
int i = 0;
char s[4];
strcpy(s, (char *) &i);
```

True

False

Incorrect

Question 14

0 / 3 pts

```
int i = 0xdeadbeef;
char s[4];
strcpy(s, (char *) &i);
```

True

False

Question 15

3 / 3 pts

```
int i = 0x00baff1e;
char s[4];
strcpy(s, (char *) &i);
```

- True
- False

Incorrect

Question 16

0 / 3 pts

```
int i, *p;
p = malloc(3);
for (i = 0; i < 3; i++) {
  p[i] = i;
}
```

- True
- False

Question 17

3 / 3 pts

```
int *s = "Hello world", *p;
p = s + 1;
printf("%s\n", p);
```

- True
- False

Quiz Score: 70 out of 100